Agentic QA LangSmith & AI Quality Engineering Handbook

Architecture, tracing, datasets, deterministic evaluation, experiments, A/B comparison, and retrieval quality

Scope
This handbook documents what we have learned and implemented in the Agentic QA project so far, plus the immediate evaluation roadmap. Code is labeled as implemented, current baseline, or planned. The focus is on testing the AI application itself from the beginning, not only testing the software it will eventually generate.

Project stack: Python + LangGraph + LangChain + Pydantic + pytest + Playwright + LangSmith

Contents

- 1. Why AI applications need a quality system from day one
- 2. How LangSmith fits our Agentic QA architecture
- 3. Core LangSmith concepts: traces, datasets, evaluators, experiments
- 4. Setup and configuration lessons, including EU region and workspace ID
- 5. Tracing the current LangGraph application
- 6. Evaluation strategy: deterministic first, semantic judges later
- 7. Requirement Analyzer evaluation architecture
- 8. Shared deterministic evaluator code
- 9. Local terminal evaluation script
- 10. Creating the LangSmith Requirement Analyzer dataset
- 11. LangSmith target, adapter, evaluators, and case_pass
- 12. Reading LangSmith experiment results
- 13. A/B comparison in our engineering context
- 14. Repository Retrieval evaluation design
- 15. Retrieval metrics: Top-1, Precision@K, Recall@K
- 16. Retrieval evaluation code and unit tests
- 17. Creating and running the LangSmith retrieval experiment
- 18. What our real V1 retrieval runs taught us
- 19. Measuring V1 -> V2 -> V3 instead of relying on intuition
- 20. Future evaluation: LLM-as-a-judge, MCP/tool trajectories, generated tests
- 21. CI/CD and regression gates
- 22. Troubleshooting and debugging checklist
- 23. Recommended next steps
- Appendix A. Evaluation-oriented repository layout
- Appendix B. Glossary

1. Why AI applications need a quality system from day one

Traditional software is largely deterministic: the same input, code version, and environment are expected to produce the same result. LLM-based applications add probabilistic behavior, prompt sensitivity, model-version changes, retrieval variation, tool-selection decisions, and multi-step trajectories. That means ordinary unit tests are necessary but not sufficient.

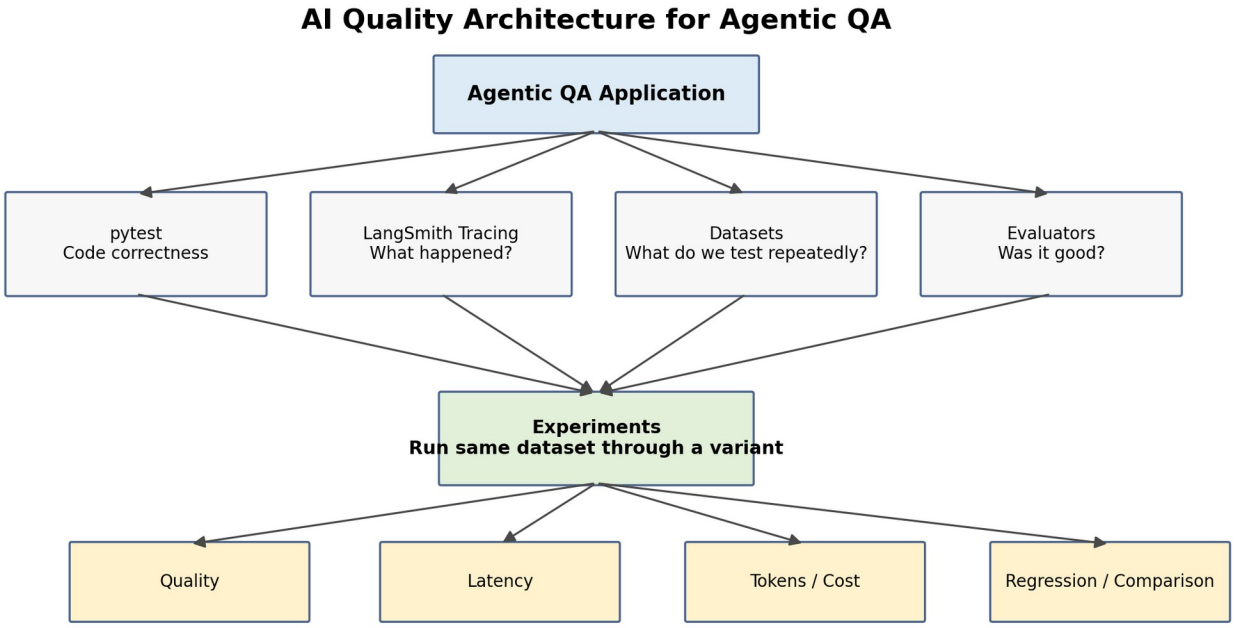


Figure 1. Quality architecture we are building around the Agentic QA application.

Core principle

Every important AI component should have its own observable behavior, reusable evaluation dataset, explicit metrics, and regression history. Full-agent quality is built on component quality.

Question	Best tool in our current stack
Does Python code behave correctly?	pytest + type checking + Pydantic
What happened during an AI run?	LangSmith tracing
Which examples should be rerun repeatedly?	LangSmith dataset
Did the output satisfy our expectations?	Deterministic evaluator or later LLM judge
How did model/prompt/retriever version A compare with B?	LangSmith experiment comparison
Why did a failed case fail?	Open experiment row -> inspect trace

2. How LangSmith fits our Agentic QA architecture

LangSmith is not replacing pytest, LangGraph, or our own evaluation logic. It provides observability and an experiment platform around them. We deliberately keep evaluation rules in our Python code so they remain testable, version-controlled, and reusable outside LangSmith.

Our quality stack

```
Agentic QA application
    |
    +-- pytest / mypy / Pydantic
    |       -> deterministic software correctness
    |
    +-- LangSmith tracing
    |       -> what happened during execution
    |
    +-- evaluation datasets
    |       -> repeatable AI regression cases
    |
    +-- shared Python evaluators
    |       -> explicit quality rules
    |
    +-- LangSmith experiments
            -> execute variants, record metrics, latency, traces
```

We considered Langfuse and DeepEval. Langfuse is attractive for vendor-neutral/self-hosted observability. DeepEval is attractive as a pytest-style AI evaluation framework. For this project, LangSmith is the simplest first platform because the orchestration layer is already LangGraph/LangChain. We can add another framework later only if it solves a real unmet requirement.

3. Core LangSmith concepts

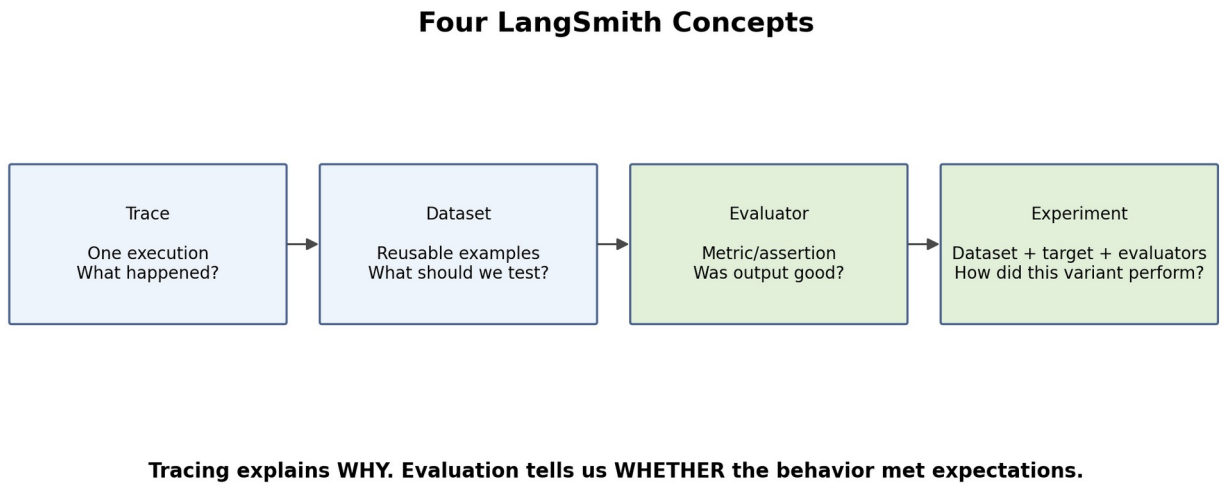


Figure 2. Trace, Dataset, Evaluator, and Experiment are different concepts.

Trace

One execution of the application. It answers: what happened, which operations ran, what inputs/outputs were observed, how long they took, and where an error occurred.

Dataset

A reusable collection of evaluation examples. It answers: what cases should we rerun whenever a model, prompt, retriever, or agent policy changes?

Evaluator

A function or judge that converts actual output + reference expectations into one or more quality scores.

Experiment

A run of a target implementation over a dataset with evaluators. It produces comparable metrics and links each evaluation result back to its trace.

> **Important distinction**
> Tracing explains WHY a run behaved the way it did. Evaluation tells us WHETHER it met the expected quality bar.

4. Setup and configuration lessons

We encountered two useful real-world configuration problems: .env loading and region/workspace authorization. They are worth documenting because observability tooling is only useful if its configuration is deterministic.

4.1 Environment variables

.env for our EU LangSmith workspace

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=...
LANGSMITH_PROJECT=agentic-qa-dev
LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com
LANGSMITH_WORKSPACE_ID=<actual-workspace-id>
```

The project already used Pydantic Settings, but Pydantic reading values from .env does not necessarily mean arbitrary libraries see those values through os.environ. LangSmith automatic tracing reads LANGSMITH_* environment variables. We therefore added python-dotenv loading so the process environment contains them.

Load .env into process environment

```
from dotenv import load_dotenv

load_dotenv()
```

> **Debugging lesson**
> OpenAI settings can work through Pydantic while LangSmith tracing still sees None. Verify os.getenv("LANGSMITH_TRACING") rather than assuming .env was exported.

Safe verification without printing the secret

```
import os
from dotenv import load_dotenv

load_dotenv()

print("TRACING:", os.getenv("LANGSMITH_TRACING"))
print("PROJECT:", os.getenv("LANGSMITH_PROJECT"))
print("ENDPOINT:", os.getenv("LANGSMITH_ENDPOINT"))
print("WORKSPACE:", os.getenv("LANGSMITH_WORKSPACE_ID"))
print("HAS KEY:", bool(os.getenv("LANGSMITH_API_KEY")))
```

4.2 Region

Our LangSmith UI is eu.smith.langchain.com, therefore our ingestion endpoint must be the EU API endpoint. When we initially sent traces to the default US endpoint, LangSmith returned HTTP 403.

LangSmith UI	Region	API endpoint
smith.langchain.com	GCP US	https://api.smith.langchain.com
eu.smith.langchain.com	GCP EU	https://eu.api.smith.langchain.com
apac.smith.langchain.com	GCP APAC	https://apac.api.smith.langchain.com
aws.smith.langchain.com	AWS US	https://aws.api.smith.langchain.com

4.3 Organization ID is not Workspace ID

We initially copied the Organization ID into LANGSMITH_WORKSPACE_ID. These are different concepts. The workspace ID must be the actual workspace/tenant identifier. The practical symptom was repeated 403 Forbidden ingestion errors even after the environment variables were present.

5. Tracing the current LangGraph application

Once tracing is enabled, we can inspect a requirement run as a hierarchy rather than only a final JSON state.

Current analyzer trace concept

```
QAState
  requirement
      |
      v
analyze_requirement
      |
      v
LLMRequirementAnalyzer.analyze()
      |
      v
ChatOpenAI
      |
      v
RequirementAnalysis
      |
      v
QAState.requirement_analysis
```

When reading a trace, inspect Input, Output, child runs, latency, token/model metadata, and errors. If an experiment later reports a failed evaluator score, the trace is where we diagnose whether the issue originated in the prompt, model behavior, retrieval context, or downstream processing.

5.1 Metadata and tags

We should tag workflow runs with version information so historical traces remain useful after code changes.

Recommended workflow metadata

```
from langchain_core.runnables import RunnableConfig

config: RunnableConfig = {
    "run_name": "agentic_qa_workflow",
    "tags": ["dev", "retrieval-v1"],
    "metadata": {
        "model": settings.llm_model,
        "retrieval_version": "v1-keyword",
        "application": "agentic-qa",
    },
}

result = workflow.invoke(
    {"requirement": requirement},
    config=config,
)
```

Recommended next instrumentation

Trace architectural operations, not every helper. Good spans: requirement analysis, repository retrieval, AST lookup, MCP action, generated-test execution. Avoid turning _build_query_terms(), _score_file(), etc. into separate spans unless debugging a specific issue.

6. Evaluation strategy: deterministic first, semantic judges later

A common AI-testing failure is using an LLM judge for everything. We deliberately use deterministic checks whenever correctness can be expressed mechanically, because deterministic evaluators are cheaper, reproducible, explainable, and easy to unit-test.

Quality question	Evaluator type
interaction_surface == api?	Deterministic code
HTTP 400 preserved in expected outcome?	Deterministic code
Required artifact retrieved in top K?	Deterministic information-retrieval metric
Did the analyzer invent unsupported assumptions?	Potential LLM judge later
Is a TestDesign risk-based and sufficient?	Human rubric + LLM judge later
Did the agent choose a sensible tool trajectory?	Trajectory evaluator later

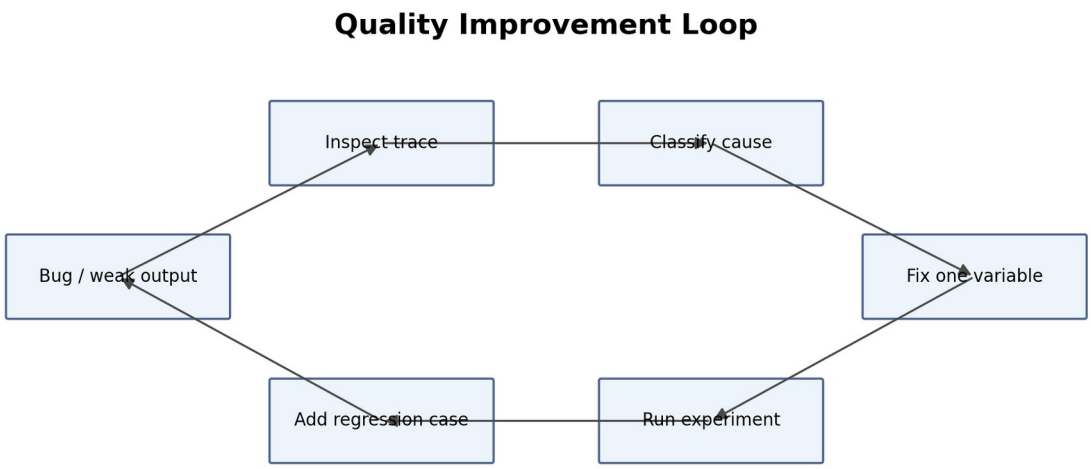


Figure 3. A useful failure becomes a permanent regression case.

7. Requirement Analyzer evaluation architecture

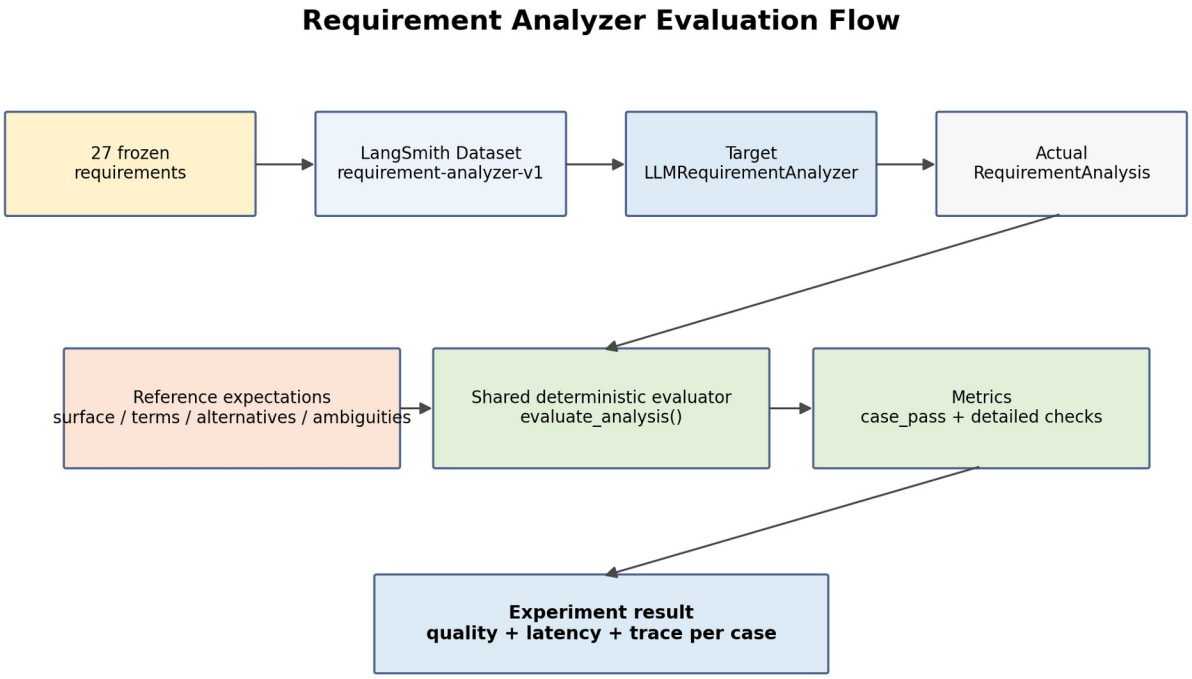


Figure 4. The first component-level AI regression suite in this project.

We already had a 27-case Requirement Analyzer regression suite before LangSmith. LangSmith did not replace it. Instead we moved the reusable evaluation rules into a shared module and used the same logic in both the terminal evaluator and LangSmith experiments.

Current baseline

The 27-case dataset is treated as a frozen regression set. A green 100% result means all configured deterministic checks passed; it does not mean the model is universally 100% accurate.

8. Shared deterministic evaluator code

The refactor moved CheckResult, term matching, alternative matching, and evaluate_analysis() out of scripts/evaluate_requirement_analyzer.py into src/agentic_qa/evals/requirements/evaluator.py. This prevents the local runner and LangSmith evaluator from drifting apart.

src/agentic_qa/evals/requirements/evaluator.py - reusable helpers

```
from dataclasses import dataclass

from agentic_qa.evals.requirements.cases import (
    RequirementEvaluationCase,
)
from agentic_qa.models import RequirementAnalysis


@dataclass
class CheckResult:
    name: str
    passed: bool
    message: str


def contains_terms(
    values: list[str],
    terms: list[str],
) -> bool:
    text = " ".join(values).lower()

    return all(
        term.lower() in text
        for term in terms
    )


def contains_alternatives(
    values: list[str],
    alternatives: list[list[str]],
) -> bool:
    text = " ".join(values).lower()

    return all(
        any(
            alternative.lower() in text
            for alternative in group
        )
        for group in alternatives
    )
```

Core deterministic evaluator

```
def evaluate_analysis(
    analysis: RequirementAnalysis,
    case: RequirementEvaluationCase,
) -> list[CheckResult]:
    results: list[CheckResult] = []

    results.append(
        CheckResult(
            name="interaction_surface",
            passed=(
                analysis.interaction_surface
                == case.expected_surface
            ),
```

```
            message=(
                f"expected={case.expected_surface.value}, "
                f"actual={analysis.interaction_surface.value}"
            ),
        )
    )

    if case.required_condition_terms:
        results.append(
            CheckResult(
                name="condition_terms",
                passed=contains_terms(
                    analysis.conditions,
                    case.required_condition_terms,
                ),
                message=(
                    "required terms="
                    f"{case.required_condition_terms}"
                ),
            )
        )

    if case.required_condition_alternatives:
        results.append(
            CheckResult(
                name="condition_concepts",
                passed=contains_alternatives(
                    analysis.conditions,
                    case.required_condition_alternatives,
                ),
                message=(
                    "accepted alternatives="
                    f"{case.required_condition_alternatives}"
                ),
            )
        )

    if case.required_outcome_terms:
        results.append(
            CheckResult(
                name="outcome_terms",
                passed=contains_terms(
                    analysis.expected_outcomes,
                    case.required_outcome_terms,
                ),
                message=(
                    "required terms="
                    f"{case.required_outcome_terms}"
                ),
            )
        )

    if case.required_outcome_alternatives:
        results.append(
            CheckResult(
                name="outcome_concepts",
                passed=contains_alternatives(
                    analysis.expected_outcomes,
                    case.required_outcome_alternatives,
                ),
                message=(
                    "accepted alternatives="
                    f"{case.required_outcome_alternatives}"
                ),
            )
        )

    if case.require_ambiguities is not None:
        has_ambiguities = bool(analysis.ambiguities)

        results.append(
            CheckResult(
                name="ambiguities",
                passed=(
                    has_ambiguities
                    == case.require_ambiguities
                ),
                message=(
```

```
                    "expected ambiguities="
                    f"{case.require_ambiguities}, "
                    "actual count="
                    f"{len(analysis.ambiguities)}"
                ),
            )
        )

    return results
```

required_*_terms represent stable facts such as HTTP codes. required_*_alternatives represent one semantic concept with acceptable lexical variants. This is how we avoid false failures from harmless wording differences while still preserving explicit contracts.

9. Local terminal evaluation script

The terminal script remains useful for fast local runs and JSON artifacts. Its responsibility is orchestration, reporting, and file output - not evaluation rules.

Cleaned local runner - abbreviated printing but full architecture

```
import json
from pathlib import Path

from langchain_openai import ChatOpenAI

from agentic_qa.config import settings
from agentic_qa.evals.requirements.cases import CASES
from agentic_qa.evals.requirements.evaluator import evaluate_analysis
from agentic_qa.requirements.llm_analyzer import LLMRequirementAnalyzer


def main() -> None:
    if settings.openai_api_key is None:
        raise RuntimeError("OPENAI_API_KEY is not configured.")

    if not settings.llm_model:
        raise RuntimeError("LLM_MODEL is not configured.")

    model = ChatOpenAI(
        model=settings.llm_model,
        api_key=settings.openai_api_key,
    )

    analyzer = LLMRequirementAnalyzer(model=model)

    total_checks = 0
    passed_checks = 0
    machine_results: list[dict[str, object]] = []

    for case in CASES:
        analysis = analyzer.analyze(case.requirement)

        checks = evaluate_analysis(
            analysis=analysis,
            case=case,
        )

        for check in checks:
            total_checks += 1
            if check.passed:
                passed_checks += 1

        machine_results.append(
            {
                "case": case.name,
                "requirement": case.requirement,
                "analysis": analysis.model_dump(mode="json"),
                "checks": [
                    {
```

```
                        "name": check.name,
                        "passed": check.passed,
                        "message": check.message,
                    }
                    for check in checks
                ],
            }
        )

    score = passed_checks / total_checks if total_checks else 0.0

    output_dir = Path("evaluation_results")
    output_dir.mkdir(parents=True, exist_ok=True)

    with (output_dir / "latest.json").open("w", encoding="utf-8") as file:
        json.dump(
            {
                "model": settings.llm_model,
                "passed_checks": passed_checks,
                "total_checks": total_checks,
                "score": score,
                "cases": machine_results,
            },
            file,
            indent=2,
        )


if __name__ == "__main__":
    main()
```

10. Creating the LangSmith Requirement Analyzer dataset

We upload our existing CASES into a LangSmith dataset named requirement-analyzer-v1. The reference output intentionally does not contain an exact expected RequirementAnalysis object. It contains evaluation constraints, because exact natural-language phrasing would be too brittle.

scripts/create_langsmith_requirement_dataset.py

```
from langsmith import Client

from agentic_qa.evals.requirements.cases import CASES

DATASET_NAME = "requirement-analyzer-v1"


def main() -> None:
    client = Client()

    if client.has_dataset(dataset_name=DATASET_NAME):
        print(f"Dataset '{DATASET_NAME}' already exists. Skipping creation.")
        return

    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="Regression dataset for the Agentic QA Requirement Analyzer.",
        metadata={
            "component": "requirement_analyzer",
            "version": "v1",
        },
    )

    examples = []

    for case in CASES:
        examples.append(
            {
                "inputs": {
                    "requirement": case.requirement,
                },
```

```
                "outputs": {
                    "expected_surface": case.expected_surface.value,
                    "required_condition_terms": case.required_condition_terms,
                    "required_condition_alternatives": case.required_condition_alternatives,
                    "required_outcome_terms": case.required_outcome_terms,
                    "required_outcome_alternatives": case.required_outcome_alternatives,
                    "require_ambiguities": case.require_ambiguities,
                },
                "metadata": {
                    "case_name": case.name,
                },
            }
        )

    client.create_examples(
        dataset_id=dataset.id,
        examples=examples,
    )


if __name__ == "__main__":
    main()
```

Why this reference format?
We want to test semantic contracts such as surface=api, required HTTP code 400, and accepted condition concepts. We do not want exact string equality against a full LLM-generated object.

11. LangSmith target, adapter, evaluators, and case_pass

11.1 Target function

System under test for the experiment

```
def target(
    inputs: dict[str, Any],
) -> dict[str, Any]:
    analysis = analyzer.analyze(
        inputs["requirement"]
    )

    return {
        "analysis": analysis.model_dump(
            mode="json"
        ),
    }
```

11.2 Why reconstruct a RequirementEvaluationCase?

LangSmith stores JSON-compatible dictionaries. Our shared evaluator expects a RequirementEvaluationCase Pydantic object. The adapter converts the dataset reference dictionary back into the domain type expected by evaluate_analysis(). This is serialization/deserialization plumbing, not a second evaluation engine.

LangSmith adapter inside evaluate_requirement_analyzer_langsmith.py

```
def case_from_reference_outputs(
    inputs: dict[str, Any],
    reference_outputs: dict[str, Any],
) -> RequirementEvaluationCase:
    return RequirementEvaluationCase(
        name="langsmith_case",
```

```
        requirement=inputs["requirement"],
        expected_surface=InteractionSurface(
            reference_outputs["expected_surface"]
        ),
        required_condition_terms=reference_outputs.get(
            "required_condition_terms", []
        ),
        required_condition_alternatives=reference_outputs.get(
            "required_condition_alternatives", []
        ),
        required_outcome_terms=reference_outputs.get(
            "required_outcome_terms", []
        ),
        required_outcome_alternatives=reference_outputs.get(
            "required_outcome_alternatives", []
        ),
        require_ambiguities=reference_outputs.get(
            "require_ambiguities"
        ),
    )
```

11.3 Detailed metrics evaluator

One evaluator emits multiple detailed feedback metrics

```
def requirement_analysis_evaluator(
    inputs: dict[str, Any],
    outputs: dict[str, Any],
    reference_outputs: dict[str, Any],
) -> list[dict[str, Any]]:
    analysis = RequirementAnalysis.model_validate(
        outputs["analysis"]
    )

    case = case_from_reference_outputs(
        inputs,
        reference_outputs,
    )

    checks = evaluate_analysis(
        analysis=analysis,
        case=case,
    )

    return [
        {
            "key": check.name,
            "score": check.passed,
            "comment": check.message,
        }
        for check in checks
    ]
```

11.4 Overall case_pass evaluator

Overall per-case regression signal

```
def case_pass_evaluator(
    inputs: dict[str, Any],
    outputs: dict[str, Any],
    reference_outputs: dict[str, Any],
) -> dict[str, Any]:
    analysis = RequirementAnalysis.model_validate(
        outputs["analysis"]
    )

    case = case_from_reference_outputs(
        inputs,
        reference_outputs,
    )
```

```
    checks = evaluate_analysis(analysis, case)
    passed = all(check.passed for check in checks)
    failed_checks = [check.name for check in checks if not check.passed]

    return {
        "key": "case_pass",
        "score": passed,
        "comment": (
            "all checks passed"
            if passed
            else "failed checks: " + ", ".join(failed_checks)
        ),
    }
```

11.5 Running the experiment

LangSmith experiment

```
results = client.evaluate(
    target,
    data=DATASET_NAME,
    evaluators=[
        requirement_analysis_evaluator,
        case_pass_evaluator,
    ],
    experiment_prefix="requirement-analyzer",
    max_concurrency=2,
    metadata={
        "model": settings.llm_model,
        "component": "requirement_analyzer",
        "analyzer_version": "v1",
    },
)
```

12. Reading LangSmith experiment results

The experiment UI shows Input, Reference Outputs, actual Outputs, evaluator columns, latency, and links to traces. In our first successful experiment we confirmed columns such as case_pass, interaction_surface, condition_*, outcome_*, and ambiguities.

Cell value	Meaning
1.00	Evaluator ran and passed
0.00	Evaluator ran and failed
No feedback	That metric was not applicable / evaluator did not emit it for this case

For example, require_ambiguities=None means the ambiguities evaluator is intentionally skipped for that case. No feedback is therefore not a failure.

Failure workflow

case_pass = 0 -> locate the red detailed metric -> open row -> compare input/reference/actual output -> open trace -> diagnose prompt/model/data/retrieval cause.

13. A/B comparison in our engineering context

Here A/B comparison means an offline controlled experiment, not splitting live website traffic. We run the same frozen dataset through two variants, change one important variable, and compare metrics.

Controlled A/B comparison

```
Same 27 requirements
        |
   +----+----+
   |         |
Variant A  Variant B
model X    model X
prompt v1  prompt v2
   |         |
   +----+----+
        |
 same evaluators
        |
 compare quality / latency / cost
```

Good comparison	Why
A = model X + prompt v1; B = model Y + prompt v1	Measures model change
A = model X + prompt v1; B = model X + prompt v2	Measures prompt change
A = V1 retriever; B = V2 retriever on same dataset	Measures retrieval architecture change

Avoid confounding

Do not change model, prompt, temperature, retrieval logic, and dataset simultaneously. If B improves, you would not know which change caused it.

14. Repository Retrieval evaluation design

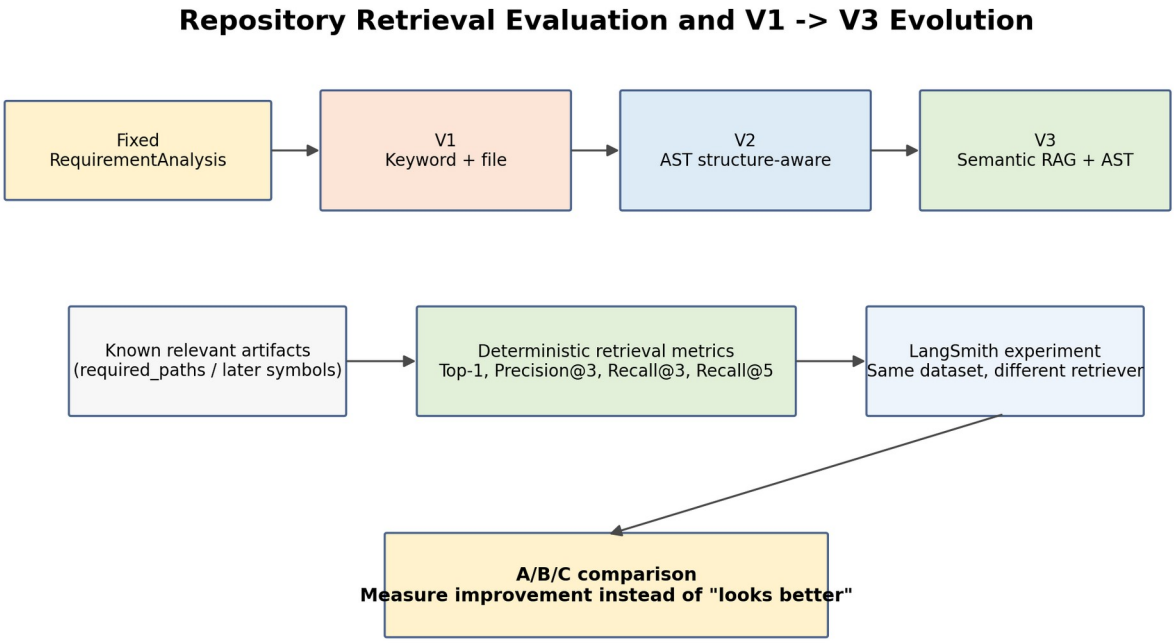


Figure 5. Retrieval evaluation isolates the retriever and keeps the dataset stable across V1/V2/V3.

The critical design choice is to feed a fixed RequirementAnalysis into the retriever evaluation. If we start from raw requirement text, a Requirement Analyzer mistake could cause a retrieval failure and we would blame the wrong component.

Component isolation

```
Wrong for component evaluation:
Raw requirement -> analyzer -> retriever -> score

Preferred:
Known RequirementAnalysis -> retriever -> score
```

Our first retrieval dataset uses the three Todo cases already tested manually: complete existing todo, create todo, and mark existing task as done. The third case deliberately uses different vocabulary (task/done vs todo/complete) to expose lexical weakness.

15. Retrieval metrics: Top-1, Precision@K, Recall@K

15.1 Top-1 relevant

Boolean metric: is the first retrieved artifact one of the known relevant artifacts? This is useful but insufficient because it says nothing about the rest of the context.

15.2 Precision@3

Among the first 3 retrieved artifacts, what fraction is relevant? Precision measures context cleanliness. Irrelevant context wastes tokens and may confuse generation.

```
Top 3 = [A relevant, D irrelevant, B relevant]
Precision@3 = 2 / 3 = 0.67
```

15.3 Recall@3 / Recall@5

Of all known relevant artifacts, what fraction was found within the first K results? Recall matters because missing an existing Page Object or fixture can cause generated tests to invent methods or duplicate setup.

```
Relevant = {A, B, C}
Top 3   = [A, D, B]
Recall@3 = 2 / 3 = 0.67

Top 5   = [A, D, B, E, C]
Recall@5 = 3 / 3 = 1.00
```

Interpretation

Recall answers "did we miss useful context?" Precision answers "how much useless context did we inject?" Our generation agent needs both.

16. Retrieval evaluation code and unit tests

16.1 Evaluation cases

src/agentic_qa/evals/retrieval/cases.py

```
from pydantic import BaseModel, Field

from agentic_qa.models import InteractionSurface, RequirementAnalysis


class RetrievalEvaluationCase(BaseModel):
    name: str
    analysis: RequirementAnalysis
    required_paths: list[str] = Field(default_factory=list)


CASES = [
    RetrievalEvaluationCase(
        name="complete_existing_todo",
        analysis=RequirementAnalysis(
            feature="Todo management",
            operation="Complete an existing todo",
            objective="Verify that a user can complete a todo that already exists.",
            actors=["User"],
            conditions=["A todo already exists."],
            expected_outcomes=[
                "The user is able to complete the existing todo."
            ],
            ambiguities=[],
            interaction_surface=InteractionSurface.UNKNOWN,
        ),
        required_paths=[
            "tests/test_todo_completion.py",
            "pages/todo_page.py",
```

```
            "tests/conftest.py",
        ],
    ),
    # create_todo ...
    # mark_task_as_done ...
]
```

16.2 Metrics implementation

src/agentic_qa/evals/retrieval/evaluator.py

```
from dataclasses import dataclass

from agentic_qa.retrieval.models import RepositoryContext


@dataclass
class RetrievalMetrics:
    top1_relevant: bool
    precision_at_3: float
    recall_at_3: float
    recall_at_5: float


def calculate_retrieval_metrics(
    context: RepositoryContext,
    required_paths: list[str],
) -> RetrievalMetrics:
    retrieved_paths = [item.path for item in context.items]
    required = set(required_paths)

    return RetrievalMetrics(
        top1_relevant=bool(
            retrieved_paths
            and retrieved_paths[0] in required
        ),
        precision_at_3=_precision_at_k(
            retrieved_paths, required, 3
        ),
        recall_at_3=_recall_at_k(
            retrieved_paths, required, 3
        ),
        recall_at_5=_recall_at_k(
            retrieved_paths, required, 5
        ),
    )


def _precision_at_k(
    retrieved_paths: list[str],
    relevant_paths: set[str],
    k: int,
) -> float:
    if k <= 0:
        raise ValueError("k must be greater than zero.")

    top_k = retrieved_paths[:k]
    hits = sum(path in relevant_paths for path in top_k)
    return hits / k


def _recall_at_k(
    retrieved_paths: list[str],
    relevant_paths: set[str],
    k: int,
) -> float:
    if not relevant_paths:
        return 1.0

    top_k = retrieved_paths[:k]
    hits = sum(path in relevant_paths for path in top_k)
    return hits / len(relevant_paths)
```

16.3 Unit-test the evaluator before trusting LangSmith

tests/unit/test_retrieval_evaluator.py

```
def test_calculates_retrieval_metrics() -> None:
    context = RepositoryContext(
        query_terms=[],
        items=[
            make_item("a.py"),
            make_item("unrelated.py"),
            make_item("b.py"),
            make_item("c.py"),
        ],
    )

    metrics = calculate_retrieval_metrics(
        context=context,
        required_paths=["a.py", "b.py", "c.py"],
    )

    assert metrics.top1_relevant is True
    assert metrics.precision_at_3 == 2 / 3
    assert metrics.recall_at_3 == 2 / 3
    assert metrics.recall_at_5 == 1.0
```

Testing principle

pytest tests the evaluator itself. LangSmith uses the evaluator after we trust its formulas. Evaluation code is production code for our quality system and deserves its own tests.

17. Creating and running the LangSmith retrieval experiment

17.1 Dataset creation

scripts/create_langsmith_retrieval_dataset.py

```
from langsmith import Client
from agentic_qa.evals.retrieval.cases import CASES

DATASET_NAME = "repository-retrieval-v1"


def main() -> None:
    client = Client()

    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="Repository retrieval regression dataset for Agentic QA.",
        metadata={
            "component": "repository_retrieval",
            "dataset_version": "v1",
        },
    )

    examples = [
        {
            "inputs": {
                "analysis": case.analysis.model_dump(mode="json"),
            },
            "outputs": {
                "required_paths": case.required_paths,
            },
            "metadata": {
                "case_name": case.name,
            },
        }
        for case in CASES
```

```
    ]

    client.create_examples(
        dataset_id=dataset.id,
        examples=examples,
    )
```

17.2 Target

V1 retrieval target

```
provider = KeywordRepositoryContextProvider(
    repository_path=settings.target_repository,
    top_k=8,
)


def target(
    inputs: dict[str, Any],
) -> dict[str, Any]:
    analysis = RequirementAnalysis.model_validate(
        inputs["analysis"]
    )

    context = provider.retrieve(analysis)

    return {
        "repository_context": context.model_dump(
            mode="json"
        )
    }
```

17.3 LangSmith retrieval evaluator

Deterministic retrieval feedback

```
def retrieval_evaluator(
    outputs: dict[str, Any],
    reference_outputs: dict[str, Any],
) -> list[dict[str, Any]]:
    context = RepositoryContext.model_validate(
        outputs["repository_context"]
    )

    required_paths = reference_outputs["required_paths"]

    metrics = calculate_retrieval_metrics(
        context=context,
        required_paths=required_paths,
    )

    return [
        {"key": "top1_relevant", "score": metrics.top1_relevant},
        {"key": "precision_at_3", "score": metrics.precision_at_3},
        {"key": "recall_at_3", "score": metrics.recall_at_3},
        {"key": "recall_at_5", "score": metrics.recall_at_5},
    ]
```

17.4 Experiment

```
results = client.evaluate(
    target,
    data=DATASET_NAME,
    evaluators=[retrieval_evaluator],
    experiment_prefix="repository-retrieval-v1",
    metadata={
        "retriever": "keyword",
        "retrieval_version": "v1",
```

```
    },
)
```

18. What our real V1 retrieval runs taught us

We ran three requirements through the real application before building the retrieval dataset. These runs give us empirical reasons to build V2 and V3.

Requirement	Observed V1 behavior	Lesson
Complete an existing todo	Correct completion test ranked first; Page Object and fixture found	Keyword retrieval works well when vocabulary aligns
Create a new todo	Relevant Page Object/test found, but analyzer changed "todo" to "todo list"	Component boundaries matter; analyzer quality can affect downstream retrieval
Mark an existing task as done	Correct test ranked first mostly because of generic "existing"; Page Object matched "done" inside placeholder text	Correct result can be returned for the wrong lexical reason

The third case was the clearest V1 limitation. Requirement vocabulary task/mark/done does not match repository vocabulary todo/complete. V1 cannot understand those semantic equivalences. It also returns snippets around the first lexical match, which can select an irrelevant line such as the placeholder "What needs to be done?" instead of complete_todo().

Why we freeze V1

Do not spend excessive effort adding hand-written synonyms, stemming, or complex scoring. V1 has proven the interface and exposed its limitations. V2 should solve structure; V3 should solve semantic mismatch.

19. Measuring V1 -> V2 -> V3 instead of relying on intuition

The retrieval dataset should remain stable while the implementation changes. This turns architecture evolution into controlled experiments.

Planned experiment series

```
Dataset: repository-retrieval-v1

Experiment A:
    KeywordRepositoryContextProvider
    -> file-level keyword retrieval

Experiment B:
    StructuredRepositoryContextProvider
    -> AST / symbol-level retrieval

Experiment C:
    HybridRepositoryContextProvider
```

```
    -> semantic RAG + AST exact lookup
```

Version	Expected improvement	Still weak at
V1 keyword/file	Basic file discovery	Semantic mismatch, symbol precision, snippet quality
V2 AST structure-aware	Methods/tests/fixtures as retrieval units; better code snippets	Synonyms and conceptual vocabulary mismatch
V3 semantic RAG + AST	Semantic similarity + exact symbol verification	Higher cost/complexity; needs stronger eval dataset

When V2 returns symbols rather than only files, each symbol should still preserve source path metadata. That lets us keep the current file-level metrics for backwards comparison while adding symbol-level expected artifacts later.

20. Future evaluation: LLM judges, MCP/tool trajectories, generated tests

20.1 LLM-as-a-judge

Use only when the question is genuinely semantic or subjective. Examples: whether ambiguities are useful, whether a TestDesign is sufficient and risk-based, whether generated code preserves business intent, or whether a repair changed the test only to make it pass.

Safe adoption sequence

Human-reviewed examples -> explicit rubric -> LLM judge -> compare judge decisions against human labels -> only then use the judge as a trusted metric.

20.2 Agent trajectory evaluation

```
Potential full-agent trajectory checks:
- Did retrieval happen before code generation?
- Was MCP used when live UI knowledge was necessary?
- Did the agent reuse an existing Page Object instead of inventing one?
- Did it call AST exact lookup before referencing an uncertain method?
- Did it classify a product defect instead of rewriting a valid test?
- Did it retry only when repair was justified?
```

20.3 Generated test quality

Generated automation eventually needs deterministic code checks (parses, imports, pytest collection, lint/mypy, locator policy, no hard sleeps), behavioral execution, and semantic/risk-based review. A passing generated test is not automatically a correct test.

21. CI/CD and regression gates

Once datasets become stable, evaluation should be part of change validation. We should not immediately block every commit on probabilistic metrics; first collect baseline distributions and identify stable thresholds.

```
Suggested progression:
1. pytest + mypy + ruff -> hard CI gates now
2. deterministic Requirement Analyzer experiment -> report on PR
3. retrieval metrics -> report V1/V2 comparison
4. after stable baselines, fail CI on severe regression thresholds
5. semantic/LLM judge metrics remain advisory until calibrated
```

Useful experiment metadata includes Git commit, model, prompt version, retrieval version, dataset version, and environment. This makes regressions reproducible rather than anecdotal.

22. Troubleshooting and debugging checklist

Symptom	Likely cause / check
LANGSMITH_* values are None	python-dotenv not loaded or environment not exported
403 posting to api.smith.langchain.com while UI is EU	wrong region endpoint
403 after adding workspace ID	organization ID used instead of workspace ID, or key/workspace mismatch
No feedback in an evaluator cell	metric not emitted for that example; may be expected
Everything looks green but only surface was checked	verify full reference_outputs were uploaded and all evaluator columns exist
LangSmith result differs from local evaluator	shared evaluation logic duplicated or adapter reconstructed case incorrectly
Retriever quality appears good but generated code invents methods	file-level evaluation is too coarse; add symbol-level V2 metrics

23. Recommended next steps

1. Implement and run the repository-retrieval-v1 LangSmith dataset/experiment using the three frozen Todo cases.
2. Record the V1 baseline: Top-1, Precision@3, Recall@3, Recall@5, latency.
3. Implement V2 AST structure-aware indexing for classes, methods, tests, and pytest fixtures - no locators yet.
4. Run the exact same retrieval dataset against V2 and compare experiments.
5. Add symbol-level expected artifacts after V2 stabilizes.
6. Implement V3 semantic retrieval only after V2 metrics show what structure alone cannot solve.

7. Then move evaluation to TestDesign, MCP exploration/tool use, code generation, and failure analysis.

Quality rule for the project
Every meaningful AI defect or weakness that we understand should become a regression example whenever practical. The goal is a growing quality corpus, not repeated prompt tweaking.

Appendix A. Evaluation-oriented repository layout

```
agentic-qa/
├── src/agentic_qa/
│   ├── graph/
│   ├── requirements/
│   ├── retrieval/
│   └── evals/
│       ├── requirements/
│       │   ├── cases.py
│       │   └── evaluator.py
│       └── retrieval/
│           ├── cases.py
│           └── evaluator.py
│
├── scripts/
│   ├── evaluate_requirement_analyzer.py
│   ├── create_langsmith_requirement_dataset.py
│   ├── evaluate_requirement_analyzer_langsmith.py
│   ├── create_langsmith_retrieval_dataset.py
│   └── evaluate_repository_retrieval_langsmith.py
│
├── tests/unit/
│   ├── test_keyword_repository_provider.py
│   ├── test_retrieval_evaluator.py
│   └── ...
│
├── sample_automation/
│   ├── pages/
│   ├── tests/
│   └── README.md
│
└── evaluation_results/
```

Responsibility separation is intentional: cases.py says WHAT quality examples exist; evaluator.py says HOW quality is measured; scripts execute locally or through LangSmith; pytest verifies the evaluation code itself.

Appendix B. Glossary

Term	Meaning in this project
Trace	One observed execution, including nested runs/spans
Run / span	One operation inside a trace, such as an LLM call or retriever
Dataset	Frozen/reusable evaluation examples
Reference outputs	Expected constraints or labels attached to a

	dataset example
Target	Implementation under evaluation
Evaluator	Function/judge producing one or more quality metrics
Experiment	Target executed against dataset + evaluators
A/B comparison	Offline comparison of two controlled variants on the same dataset
case_pass	All configured deterministic Requirement Analyzer checks passed for that example
Precision@K	Relevant retrieved items / K
Recall@K	Relevant retrieved items found in top K / total known relevant items
LLM-as-a-judge	Model-based semantic evaluator, to be calibrated against human judgments
Trajectory evaluation	Evaluation of sequence of agent/tool decisions, not only final output

End of handbook - current quality baseline before Repository Retrieval V2

Evaluation documentation revision — 18 October 2026

Documentation update | 18 October 2026

STATUS: The original handbook chapters are preserved as historical milestones. This revision supersedes older “current” claims about 7-case retrieval baselines and the V2 retriever being planned. All results below are from the two user-provided LangSmith CSV exports; no new LangSmith evaluation was executed during this update.

Current workflow and scope

Requirement -> analyze_requirement -> retrieve_context -> inspect_symbols (planned/partly independent) -> explore_ui (MCP integration planned) -> design_test -> generate_test -> run_test -> analyze_result -> review/repair. The actively evaluated retrieval strategies are V1.2 token-aware file retrieval and V2.0 AST symbol retrieval followed by unique-file projection. V2.1 dependency expansion is proposed, not implemented.

Repository retrieval datasets

repository-retrieval-realworld-v1: frozen 7-case benchmark (4 auth, 3 article). repository-retrieval-realworld-v2: 10-case benchmark (4 auth, 3 article, 3 comment), run on the expanded sample_automation corpus. Historic v1 experiments must be retained; comparisons between algorithm versions should use the same dataset version and repository revision.

V2.0 retrieval implementation

PythonAstSymbolIndexer produces RepositorySymbolIndex. SymbolRepositoryContextProvider ranks individual symbols using exact token matches in names, parameters, parent/path and source. project_symbols_to_repository_context deduplicates ranked symbols into a RepositoryContext of unique file paths (top 8). V1.2 remains token-aware whole-file keyword retrieval. Neither retriever currently performs relationship-driven fixture/import dependency expansion.

LangSmith evaluation v2 | measured macro averages

Metric | V1.2 | V2.0 | Difference (percentage points): precision_at_3: 46.7% -> 50.0% (+3.3 pp); recall_at_3: 33.7% -> 35.3% (+1.6 pp); recall_at_5: 45.5% -> 49.8% (+4.3 pp); top1_relevant: 80.0% -> 100.0% (+20.0 pp). All 10 cases completed in both exports. Top1 improves markedly; recall remains moderate.

Per-case Recall@5

Edit article: V1.2 60% / V2.0 40%; Delete own comment: V1.2 17% / V2.0 33%; View existing comment: V1.2 20% / V2.0 20%; Delete article: V1.2 40% / V2.0 20%; Register a new user: V1.2 50% / V2.0 50%; Sign in with registered credentials: V1.2 25% / V2.0 75%; Attempt to sign up with a valid email and password while leaving the username empty: V1.2 100% / V2.0 67%; Create comment: V1.2 17% / V2.0 33%; Create article: V1.2 60% / V2.0 60%; Sign in with a valid registered email and an invalid password: V1.2 67% / V2.0 100%

Interpretation and limitation

V2.0 improves primary-artifact identification (Top1: 80% to 100%) with limited improvement in context completeness (Recall@5: 45.5% to 49.8%). Ranked results are often dominated by similar test files instead of fixtures, API client, Page Objects, and data builders. In Delete Comment, V2.0 returns only two of six required artifacts by rank 8: the test and comments fixture; the Page Object, Comment component, API client, and builder are absent. This supports evaluating dependency-aware expansion, not blindly changing weights. A 10-case benchmark is small and includes existing tests whose names echo requirements; results do not establish generalization to unseen behaviors.

V2.1 plan and evaluation contract

PROPOSED, not implemented: start from top ranked tests/symbols; inspect pytest fixture parameters, local imports and Page Object/component references; resolve exact symbol definitions with AST; add limited one-hop related artifacts; project to deduplicated files. Evaluate V2.1 against frozen v2 dataset and same corpus; retain Top1 while improving Recall@3/5/8, monitor precision and failure cases. Do not modify V1.2 or V2.0 during the comparison.

Reproducibility and snapshot discrepancy

Current user-reported LangSmith runs targeted repository-retrieval-realworld-v2. The attached agentic-qa(2).zip snapshot still contains scripts pointing to repository-retrieval-realworld-v1 and an empty comments.py evaluation list. Treat this as a snapshot mismatch: the successful v2 experiments and CSV exports evidence newer local changes not included in that archive. Synchronize a committed repository state before any future rerun; do not claim the archive itself contains the 10-case dataset source.

Quality safeguards

Run ruff, mypy and pytest; keep fixture setup/cleanup API-backed and browser behavior UI-backed. In single-script mypy invocations, ensure the local src/ package is discoverable and typed (py.typed if distributing as PEP 561 typed package). Do not confuse mypy packaging problems with retrieval evaluation regressions. Preserve failing test evidence before any repair.

Evidence

Sources: 10-row CSV files realworld-dataset-v2-retrieval-v1.2-token-aware-48b9dd6a.csv and realworld-datadset-v2-retrieval-v2.0-symbol-token-aware-94cb88d6.csv; repository snapshot agentic-qa(2).zip; historical docs/ handbooks. Numbers are macro averages of exported feedback scores.